

opentelemetry

文档: <https://opentelemetry.io/>

go源码: <https://github.com/open-telemetry/opentelemetry-go/>

demo: <https://github.com/open-telemetry/opentelemetry-demo>

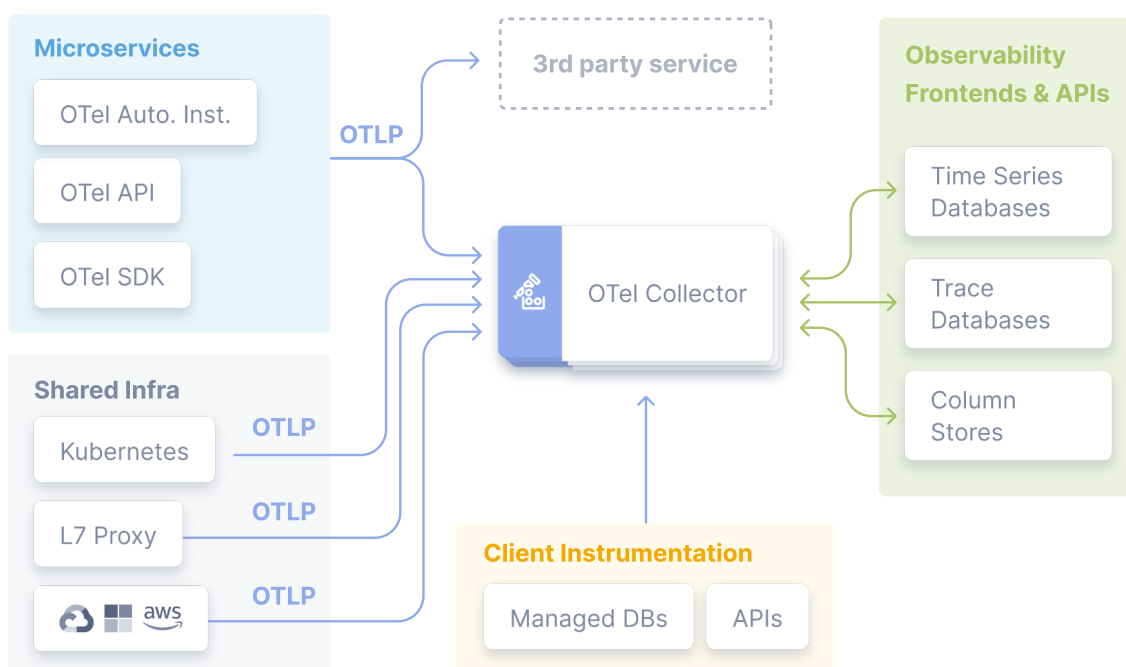
语义约定: <https://github.com/open-telemetry/semantic-conventions>

otel-collector配置文档: <https://opentelemetry.io/docs/collector/configuration/>

OpenTelemetry收集器组件库: <https://github.com/open-telemetry/opentelemetry-collector-contrib>

是什么

OpenTelemetry, 简称OTel, 是一个与供应商无关的开源可观测性框架, 用于检测、生成、收集和导出遥测数据, 如轨迹、度量、日志。OTel的目标是提供一套标准化的供应商无关SDK、API和工具, 用于接收、转换数据并将数据发送到可观测系统(监控系统)后端(即开源或商业供应商)



解决什么问题

解决各个系统之间没有统一标准的问题, 按照统一标准执行, 有助于观测系统的变更和调整。例如: zipkin、jaeger等

能做什么

1. 每种语言提供了相对应的lib库
2. 提供了中立的数据收集器, 并支持多种方式部署
3. 生成、发送、收集、处理和导出遥测数据
4. 可以通过配置将数据并行发送到多个目的地
5. 提供OpenTracing和OpenCensus项目提供垫片以向后兼容OpenTracing与OpenCensus项目

相关概念

Traces

TracerProvider

是Tracer的工厂。在大多数应用程序中，Tracer提供程序初始化一次，其生命周期与应用程序的生命周期相匹配。Tracer Provider初始化还包括Resource和Exporter初始化。这通常是使用OpenTelemetry进行跟踪的第一步

Tracer

Tracer（跟踪程序）用于创建span，其中包含有关给定操作（例如服务中的请求）所发生情况的更多信息。跟踪程序是从跟踪程序提供程序创建的

Trace Exporters

跟踪导出器将跟踪发送给消费者。此使用者可以是调试和开发时的标准输出、OpenTelemetry收集器或您选择的任何开源或供应商后端

Context Propagation（上下文传播）

上下文传播是实现分布式跟踪的核心概念。使用上下文传播，跨域可以相互关联并组装到跟踪中，而不管跨域是在哪里生成的

上下文（Context）是一个对象，它包含发送和接收服务的信息，以便将一个跨度与另一个跨度相关联，并将其与整个跟踪相关联。

传播（Propagation）是在服务和流程之间移动上下文的机制。它序列化或反序列化上下文对象，并提供要从一个服务传播的相关跟踪信息

Spans

跨度表示一个工作或操作单元。跨度是痕迹的组成部分。在OpenTelemetry中，它们包括以下信息：

1. Name（名称）
2. Parent span ID (empty for root spans)（父跨度ID，为空则表示根跨度）
3. Start and End Timestamps 开始与结束的时间戳
4. Span Context（跨度上下文）
5. Attributes（属性）
6. Span Events（事件）
7. Span Links（链接）
8. Span Status（状态）

Span Context

span context 是每个跨度上的一个不可变对象，包含以下内容：

Trace ID表示跨度所属的跟踪

Span ID 跨度的跨度ID

Trace Flags，一种包含跟踪信息的二进制编码

Trace State，可以携带特定于供应商的跟踪信息的键值对列表

span context 是与分布式上下文和Baggage一起序列化和传播的跨度的一部分。

由于“span context”包含跟踪ID，因此在创建“span link”时会使用它

Attributes

属性是包含元数据的键值对，您可以使用这些元数据对跨度进行注释，以携带有关其正在跟踪的操作的信息。

例如，如果跨度跟踪在电子商务系统中将商品添加到用户购物车的操作，则可以捕获用户的ID、要添加到购物车的商品的ID和购物车ID。

属性具有每个语言SDK实现的以下规则：

键必须是非空字符串值

值必须是非空字符串、布尔值、浮点值、整数或这些值的数组

此外，还有语义属性，这是常见操作中通常存在的元数据的已知命名约定。尽可能使用语义属性命名有助于跨系统标准化常见类型的元数据。语义约定详见文档：<https://github.com/open-telemetry/semantic-conventions>

Span Events

跨度事件可以被认为是跨度上的结构化日志消息（或注释），通常用于表示跨度持续时间内的一个有意义的奇异时间点。

例如，考虑web浏览器中的两种情况：

跟踪页面加载

表示页面何时变为交互式

跨度最适合用于第一种情况，因为它是一个有开始和结束的操作。

跨度事件最好用于跟踪第二个场景，因为它代表了一个有意义的奇异时间点

Span Links

链接的存在使您可以将一个跨度与一个或多个跨度相关联，从而暗示因果关系。例如，假设我们有一个分布式系统，其中一些操作由跟踪跟踪。

作为对其中一些操作的响应，额外的操作排队等待执行，但其执行是异步的。我们也可以通过跟踪来跟踪此后续操作。

我们希望将后续操作的跟踪与第一个跟踪相关联，但我们无法预测后续操作何时开始。我们需要将这两个轨迹关联起来，所以我们将使用跨度链接。

您可以将第一个跟踪中的最后一个跨度链接到第二个跟踪的第一个跨度。现在，它们是因果关联的。

链接是可选的，但也是将跟踪跨度相互关联的好方法

Span Status

span的状态可选值：Unset, Ok, Error。当应用程序代码中存在已知错误（例如异常）时，可将状态可以设置为“Error”

Span Kind (span 类型)

创建跨度时，它是客户端（Client）、服务器（Server）、内部（Internal）、生产者（Producer）或消费者（Consumer）之一。这种span类型为跟踪后端提供了一个关于应该如何组装跟踪的提示。根据OpenTelemetry规范，服务器跨度的父跨度通常是远程客户端跨度，而客户端跨度的子跨度通常是服务器跨度。类似地，消费者跨度的父代始终是生产者，生产者跨度的子代始终是消费者。如果未提供，则假定跨度类型为内部

1. SERVER：表示跨度涵盖同步RPC或其他远程请求的服务器端处理。此跨度通常是预期等待响应的远程CLIENT跨度的子跨度。

2. CLIENT：表示跨度描述对某个远程服务的请求。此跨度通常是远程SERVER跨度的父跨度，在收到响应之前不会结束。
3. PRODUCER：表示跨度描述异步请求的发起方。此父跨度通常会在相应的子“消费者”跨度之前结束，甚至可能在子跨度开始之前结束。在使用批处理的消息传递场景中，跟踪单个消息需要为每个要创建的消息创建一个新的生产者跨度。
4. CONSUMER：表示跨度描述异步生产者请求的子级。
5. INTERNAL：默认值。指示跨度表示应用程序中的内部操作，而不是具有远程父级或子级的操作。

Metrics

度量是在运行时捕获的关于服务的度量。从逻辑上讲，捕获其中一个测量的时刻被称为度量事件，它不仅包括测量本身，还包括捕获它的时间和相关的元数据。

应用程序和请求度量是可用性和性能的重要指标。自定义指标可以深入了解可用性指标如何影响用户体验或业务。收集的数据可用于警告停机或触发调度决策，以在高需求时自动扩大部署规模。

Meter Provider

是 Meters 的工厂。在大多数应用程序中，Meter Provider 初始化一次，其生命周期与应用程序的生命周期相匹配。Meter Provider 初始化还包括资源和导出程序初始化。这通常使用 OpenTelemetry 进行测量的第一步。

Meter

Meter 创建度量仪表，在运行时捕获有关服务的测量值。meter 是从 Meter Provider 创建的。

Metric Exporter

Metric Exporter 向消费者发送度量数据。此使用者可以是调试和开发时的标准输出、OpenTelemetry 收集器或您选择的任何开源或供应商后端。

Metric Instruments

在 OpenTelemetry 中，测量由公制仪器捕获。这种公制仪器由名称、种类、可选单位和可选说明定义。这种工具的名称、单元和描述由开发人员选择，或通过常见的语义约定（如请求或过程度量）进行定义。

语义约定详见文档：<https://github.com/open-telemetry/semantic-conventions>

仪器类型为以下六种之一：

Counter

一个随着时间积累的值——你可以把它想象成汽车上的里程表；它只会上升。

Asynchronous Counter

与 Counter 相同，但每次导出都会收集一次。如果您不能访问连续增量，而只能访问聚合值，则可以使用。

UpDownCounter

一种随着时间的推移而积累的值，但也可能再次下降。例如，队列长度会随着队列中工作项的数量而增加或减少。

Asynchronous UpDownCounter

与UpDownCounter相同，但每次导出都会收集一次。如果您不能访问连续更改，而只能访问聚合值（例如，当前队列大小），则可以使用

(Asynchronous) Gauge

测量读取时的当前值。一个例子是车辆中的燃油表。仪表总是异步的

Histogram

直方图是客户端值的集合，例如请求延迟。如果你有很多值，并且不是对每个单独的值都感兴趣，而是对这些值的统计数据感兴趣，那么直方图可能是一个不错的选择（例如，有多少请求需要少于1？）

Aggregation

除了度量工具之外，聚合的概念也是一个需要理解的重要概念。聚合是一种技术，通过该技术，将大量测量值组合成关于在时间窗口期间发生的度量事件的精确或估计统计数据。OTLP协议传输这样的聚合度量。OpenTelemetry API为每个仪器提供了一个默认聚合，可以使用视图覆盖这些聚合。

OpenTelemetry项目旨在提供可视化工具和遥测后端支持的默认聚合与请求追踪不同，metrics旨在提供汇总的统计信息。例如：

1. 报告每个协议类型的服务读取的字节总数。
2. 报告读取的字节总数和每个请求的字节数。
3. 报告系统调用的持续时间。
4. 报告请求大小以确定趋势。
5. 报告进程的CPU或内存使用情况。
6. 报告帐户的平均余额值。
7. 报告正在处理的当前活动请求

Views

视图为SDK用户提供了自定义SDK输出的度量的灵活性。他们可以自定义要处理或忽略的公制仪表。他们可以自定义聚合以及要在度量上报告的属性

Baggage

Baggage提供了一种统一的方式来存储和传播轨迹和其他信号中的信息。

在OpenTelemetry中，Baggage是跨段之间传递的上下文信息。它是一个键值存储，位于跟踪中的span上下文旁边，使在该跟踪中创建的任何span都可以使用值。

例如，假设您希望在跟踪中的每个跨度上都有一个CustomerId属性，该属性涉及多个服务；但是，CustomerId仅在一个特定服务中可用。为了实现您的目标，您可以使用OpenTelemetry Baggage在系统中传播此值。

OpenTelemetry使用上下文传播来传递Baggage，每个不同的库实现都有传播程序，可以解析并使Baggage可用，而无需显式实现它

Baggage不属于span属性的子集，因此不会自动出现在子系统span的属性中，需要明确的将信息从baggage中取出并附加到span的属性。

Resources

资源是一种特殊类型的属性，适用于流程生成的所有跨度。这些应该用于表示关于非临时进程的底层元数据，例如，进程的主机名或其实例ID

资源应该在初始化时分配给跟踪程序提供程序，并且创建时与属性非常相似

logs

日志是一种带有时间戳的文本记录，可以是结构化的（推荐的），也可以是非结构化的，带有元数据。在所有遥测信号日志中，具有最大的遗留问题。大多数编程语言都有内置的日志记录功能或众所周知的、广泛使用的日志记录库。尽管日志是一个独立的数据源，但它们也可以连接到跨度。在

OpenTelemetry中，任何不属于分布式跟踪或度量的数据都是日志。例如，事件是一种特定类型的日志。日志通常包含详细的调试/诊断信息，例如操作的输入、操作的结果以及该操作的任何支持元数据。对于跟踪和度量，OpenTelemetry采用了一种干净的设计方法，指定了一个新的API，并在多语言SDK中提供了此API的完整实现

OpenTelemetry使用日志的方法不同。由于现有的日志记录解决方案在语言和操作生态系统中广泛存在，OpenTelemetry充当了这些日志、跟踪和度量信号以及其他OpenTelemetrys组件之间的“桥梁”。事实上，由于这个原因，日志的API被称为“Logs Bridge API”

Log Appender / Bridge

作为应用程序开发人员，您不应该直接调用Logs Bridge API，因为它是为日志库作者提供的，用于构建日志附加程序/桥。相反，您只需使用首选的日志记录库，并将其配置为使用能够将日志发送到OpenTelemetry LogRecordExporter的日志附加器（或日志桥）

Logger Provider

记录器提供程序（有时称为LoggerProvider）是记录器的工厂。在大多数情况下，Logger Provider只初始化一次，其生命周期与应用程序的生命周期相匹配。Logger Provider初始化还包括Resource和Exporter初始化

是Logs Bridge API的一部分，仅当您是日志库的作者时才应使用

Logger

Logger创建日志记录。记录器是从日志提供程序创建的

是Logs Bridge API的一部分，仅当您是日志库的作者时才应使用

Log Record Exporter

日志记录导出器将日志记录发送给使用者。此使用者可以是调试和开发时的标准输出、OpenTelemetry收集器或您选择的任何开源或供应商后端

Log Record

日志记录表示事件的记录。在OpenTelemetry中，日志记录包含两种字段：

1. 具有特定类型和含义的命名顶级字段
2. 任意值和类型的资源和属性字段

日志记录顶级字段：

Timestamp	事件发生的时间
ObservedTimestamp	观察到事件的时间
TraceId	请求跟踪ID
SpanId	请求跨度ID
TraceFlags	W3C跟踪标志
SeverityText	严重性文本（也称为日志级别）
SeverityNumber	严重程度的数值
Body	日志记录的正文
Resource	描述日志的来源
InstrumentationScope	描述发出日志的作用域
Attributes	有关该事件的其他信息

详见文档：<https://opentelemetry.io/docs/specs/otel/logs/data-model/>

数据示例

1. trace

```
{
  "name": "Hello-Greetings",
  "context": {
    "trace_id": "0x5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "0x5fb397be34d26b51",
  },
  "parent_id": "0x051581bf3cb55c13",
  "start_time": "2022-04-29T18:52:58.114304Z",
  "end_time": "2022-04-29T18:52:58.114435Z",
  "attributes": {
    "http.route": "some_route1"
  },
  "events": [
    {
      "name": "hey there!",
      "timestamp": "2022-04-29T18:52:58.114561Z",
      "attributes": {
        "event_attributes": 1
      }
    },
    {
      "name": "bye now!",
      "timestamp": "2022-04-29T22:52:58.114561Z",
      "attributes": {
        "event_attributes": 1
      }
    }
  ],
}
```

```

}
{
  "name": "Hello-Salutations",
  "context": {
    "trace_id": "0x5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "0x93564f51e1abe1c2",
  },
  "parent_id": "0x051581bf3cb55c13",
  "start_time": "2022-04-29T18:52:58.114492Z",
  "end_time": "2022-04-29T18:52:58.114631Z",
  "attributes": {
    "http.route": "some_route2"
  },
  "events": [
    {
      "name": "hey there!",
      "timestamp": "2022-04-29T18:52:58.114561Z",
      "attributes": {
        "event_attributes": 1
      }
    }
  ],
}
{
  "name": "Hello",
  "context": {
    "trace_id": "0x5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "0x051581bf3cb55c13",
  },
  "parent_id": null,
  "start_time": "2022-04-29T18:52:58.114201Z",
  "end_time": "2022-04-29T18:52:58.114687Z",
  "attributes": {
    "http.route": "some_route3"
  },
  "events": [
    {
      "name": "Guten Tag!",
      "timestamp": "2022-04-29T18:52:58.114561Z",
      "attributes": {
        "event_attributes": 1
      }
    }
  ],
}

```

2. span案例

```

{
  "trace_id": "7bba9f33312b3dbb8b2c2c62bb7abe2d",
  "parent_id": "",
  "span_id": "086e83747d0e381e",
  "name": "/v1/sys/health",
  "start_time": "2021-10-22 16:04:01.209458162 +0000 UTC",
  "end_time": "2021-10-22 16:04:01.209514132 +0000 UTC",

```



```
"status_code": "STATUS_CODE_OK",
"status_message": "",
"attributes": {
  "net.transport": "IP.TCP",
  "net.peer.ip": "172.17.0.1",
  "net.peer.port": "51820",
  "net.host.ip": "10.177.2.152",
  "net.host.port": "26040",
  "http.method": "GET",
  "http.target": "/v1/sys/health",
  "http.server_name": "mortar-gateway",
  "http.route": "/v1/sys/health",
  "http.user_agent": "Consul Health Check",
  "http.scheme": "http",
  "http.host": "10.177.2.152:26040",
  "http.flavor": "1.1"
},
"events": [
  {
    "name": "",
    "message": "OK",
    "timestamp": "2021-10-22 16:04:01.209512872 +0000 UTC"
  }
]
}
```

开发

opentelemetry结构

api

为了将OpenTelemetry集成到任何项目中，API用于定义如何生成telemetry

```
go get go.opentelemetry.io/otel
go get go.opentelemetry.io/otel/trace
```

SDK

实现了opentelemetry API并符合OpenTelemetry规范

```
go get go.opentelemetry.io/otel/sdk
go get go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc
```

docker 启动一个jaeger

1. 启动容器

```
docker run -d --name jaeger \
-e COLLECTOR_ZIPKIN_HOST_PORT=:9411 \
-e COLLECTOR_OTLP_ENABLED=true \
```

```
-p 6831:6831/udp \  
-p 6832:6832/udp \  
-p 5778:5778 \  
-p 16686:16686 \  
-p 4317:4317 \  
-p 4318:4318 \  
-p 14250:14250 \  
-p 14268:14268 \  
-p 14269:14269 \  
-p 9411:9411 \  
jaegertracing/all-in-one:1.47
```

2. 浏览器访问 <http://192.168.239.154:16686>
3. jaeger api地址: <http://192.168.239.154:14268/api/traces>
4. zipkin api地址: <http://192.168.239.154:9411/api/v2/spans>
5. otel http api地址: <http://192.168.239.154:4318>
6. otel grpc api地址: <http://192.168.239.154:4317>

docker 启动一个opentelemetry-collector采集器

1. 创建配置文件

```
extensions:  
  health_check:  
    endpoint: 0.0.0.0:13133  
  pprof:  
    endpoint: 0.0.0.0:1777  
  zpages:  
    endpoint: 0.0.0.0:55679  
  
receivers:  
  # traces,metrics,logs  
  otel:  
    protocols:  
      grpc:  
        endpoint: 0.0.0.0:4317  
      http:  
        endpoint: 0.0.0.0:4318  
  
  # Collect own metrics  
  prometheus:  
    config:  
      scrape_configs:  
        - job_name: 'otel-collector'  
          scrape_interval: 10s  
          static_configs:  
            - targets: ['localhost:8888']  
  
  # traces  
  jaeger:  
    protocols:  
      grpc:  
        endpoint: 0.0.0.0:14250  
      thrift_binary:
```

```

    thrift_compact:
    thrift_http:
        endpoint: 0.0.0.0:14268

# traces
zipkin:
    endpoint: 0.0.0.0:9411

processors:
# 批处理器,可处理 traces, metrics, logs
batch:
    # 每个批次中可包含的数据包的数量,默认8192
    send_batch_size: 8192
    # 超时时间,默认200ms
    timeout: 200ms
    # 每个批次中所有数据包的大小之和,0表示不限制
    send_batch_max_size: 0
    # 设置后,此处理器将为client.Metadata中每个不同的值组合创建一个批处理程序实例
    metadata_keys:
        - host.name
        - method
    # 当metadata_keys不为空时,此设置将限制在进程生命周期内处理的元数据键值的唯一组合的数量
    # 元数据的每个不同组合都会触发在收集器中分配一个新的后台任务,该任务在进程的生命周期内运行,
    # 并且每个后台任务都保存一批挂起的最多send_batch_size记录。因此,按元数据进行批处理可以显著增加专用于批处理的内存量。
    # 不同组合的最大数量限制为配置的metadata_cardinality_limit,默认为1000以限制内存影响。
    metadata_cardinality_limit: 1000
    # memory_limiter处理器允许对内存使用情况进行定期检查,如果超过定义的限制,将开始拒绝数据并强制GC减少内存消耗
    # memory_limiter使用软内存和硬内存限制。硬限制总是高于或等于软限制
    memory_limiter:
        # 内存检查时间间隔
        check_interval: 1s
        # 进程堆目标分配的最大内存量(以MiB为单位),即硬内存限制
        limit_mib: 500
        # 内存使用量测量之间预期的最大峰值。该值必须小于limit_mib。
        # 软限制值将等于(limit_mib-spike_limit_mib)。
        # spike_limit_mib的推荐值约为limit_mib的20%
        # 即进程内存使用量的波动范围,表示允许进程短时间内增加多少内存
        spike_limit_mib: 100
        # 按百分比限制内存,固定内存设置(limit_mib)优先于百分比配置
        limit_percentage: 0
        # 按百分比设置波动范围
        spike_limit_percentage: 0

exporters:
    otlp:
        endpoint: 192.168.239.154:4317
        tls:
            insecure: true
    otlphttp:
        endpoint: http://192.168.239.154:4318

jaeger:
    endpoint: 192.168.239.154:14250
    tls:

```

```

    insecure: true
    #cert_file: cert.pem
    #key_file: cert-key.pem
zipkin:
    endpoint: http://192.168.239.154:9411/api/v2/spans
prometheus:
    endpoint: 0.0.0.0:8889
    namespace: default
service:
    pipelines:
        traces:
            receivers: [otlp,jaeger,zipkin]
            processors: [memory_limiter,batch]
            exporters: [otlp,otlphttp,jaeger,zipkin]
        metrics:
            receivers: [otlp,prometheus]
            processors: [memory_limiter,batch]
            exporters: [prometheus]
    extensions: [health_check, pprof, zpages]

```

2. 启动容器

```

docker run -d --name otl-collector \
-p 2777:1777 \
-p 9888:8888 \
-p 9889:8889 \
-p 23133:13133 \
-p 5317:4317 \
-p 5318:4318 \
-p 55680:55679 \
-p 24250:14250 \
-p 24268:14268 \
-p 19411:9411 \
-v /home/nick/work/otelcol/config.yaml:/etc/otelcol/config.yaml \
otel/opentelemetry-collector:0.81.0

```

- 1777 # pprof extension
- 8888 # Prometheus metrics exposed by the collector
- 8889 # Prometheus exporter metrics
- 13133 # health_check extension
- 4317 # OTLP gRPC receiver
- 4318 # OTLP http receiver
- 55679 # zpages extension
- 14250 # jaeger grpc
- 14268 # jaeger http
- 9411 # zipkin

3. opentelemetry采集器 contrib 版本

```
docker run -d --name otel-collector-contrib \
-p 2777:1777 \
-p 9888:8888 \
-p 9889:8889 \
-p 23133:13133 \
-p 5317:4317 \
-p 5318:4318 \
-p 55680:55679 \
-p 24250:14250 \
-p 24268:14268 \
-p 19411:9411 \
-v /home/nick/work/otelcol/config.yaml:/etc/otelcol-contrib/config.yaml \
otel/opentelemetry-collector-contrib:0.81.0
```

采集器配置

接收器 (Receiver)

用于控制数据如何进入收集器的接收器，它基于推送或拉取。通常，接收器接受指定格式的数据，将其转换为内部格式，并将其传递给适用管道中定义的处理器和导出器。默认情况下，不会配置接收器。必须配置一个或多个接收器。接收器可以支持一个或多个数据源

处理器 (Processors)

(可选) 处理器在接收数据和导出数据之间运行。默认情况下，不启用处理器。每个数据源都必须启用处理器。不是所有处理器都支持所有数据源。根据数据源，建议启用多个处理器。此外，请务必注意处理器的顺序，这很重要

导出器 (Exporters)

用于控制数据如何发送到一个或多个后端/目的地的导出器，它基于推送或拉取。默认情况下，不会配置导出器。必须配置一个或多个导出器。导出器可能支持一个或多个数据源。导出器可能会附带默认设置，但一般还需要指定目标和安全设置

连接器 (Connectors)

连接器既是导出器又是接收器。顾名思义，连接器连接两条管道：它在一条管道末端作为导出器消耗数据，在另一条管道起点作为接收器发射数据。它可以消耗和发出相同数据类型或不同数据类型的数据。连接器可以生成并发送数据来汇总消耗的数据，也可以简单地复制或路由数据。

扩展 (Extensions)

扩展主要用于不涉及处理遥测数据的任务。扩展的示例包括运行状况监视、服务发现和数据转发。扩展是可选的。

服务 (Service)

服务部分用于根据接收器、处理器、导出器和扩展部分中的配置，配置收集器中启用的组件。如果组件已配置，但未在服务部分中定义，则不会启用该组件。包括三个部分：

1. extensions, 要启用的所有扩展的列表
2. pipelines, 包括三个类型: traces、metrics、logs
3. telemetry, 是指可以配置收集器本身的metrics和logs